Planejamento Lógico TaskTracker

Nome Completo: Matheus Andrade Naves
Curso: Ciência de Dados e Machine Learning
Campus: Asa Norte
E-mail Institucional: matheus.an@sempreceub.com

---

1. Apresentação e Entendimento do Problema

O Problema a Ser Resolvido

Muitas pessoas enfrentam dificuldades para organizar suas demandas diárias, contando com anotações em papel ou mensagens dispersas em aplicativos que não são apropriados para isso. Essa falta de organização resulta em esquecimento de prazos, perda de informações essenciais, dificuldade de priorizar o que é mais urgente e uma falsa sensação de sobrecarga ou falta de produtividade, já que não há um registro claro do que foi concluído ao final do dia.

A Solução Proposta (TaskTracker)

O TaskTracker solucionará esse problema centralizando todas as pendências em um sistema digital e organizado. Ele permitirá que o usuário cadastre suas tarefas, defina uma prioridade de urgência para cada uma e visualize de forma simples tudo o que ainda precisa ser feito. Ao final, o usuário poderá marcar as tarefas como concluídas, gerando clareza e controle sobre sua rotina.

---

2. Decomposição do Problema e Dados do Sistema

Entradas de Dados (O que o usuário digita)	Saídas de Dados (O que o sistema mostra)
• Título da Tarefa • Descrição (opcional) • Prioridade (Alta, Média, Baixa) • Data de Vencimento	• Lista de Tarefas Pendentes • Mensagem: "Tarefa cadastrada com sucesso!" • Mensagem de Erro (caso um dado falte) • Resumo/Total de tarefas concluídas

Regras de Negócio do Sistema

1. **Obrigatoriedade:** O sistema não pode aceitar o cadastro de uma nova tarefa se o campo "Título" estiver vazio.
2. **Restrição de Valores:** O campo "Prioridade" só pode receber um de três valores predefinidos: Alta, Média ou Baixa.
3. **Validação de Data:** Não deve ser possível cadastrar uma tarefa com a data de vencimento no passado.

---

3. Receita Lógica do Sistema (Algoritmo)

Passo a Passo: Cadastrar Tarefa

1. O sistema solicita que o usuário insira o Título da tarefa.
2. O sistema verifica se o título não está em branco.
3. Se o título estiver em branco, exibir mensagem de erro e voltar ao passo 1.
4. O sistema solicita que o usuário escolha a Prioridade.
5. O sistema recebe as informações e armazena a nova tarefa na lista de pendências.
6. O sistema exibe a mensagem: "Tarefa cadastrada com sucesso!".

Passo a Passo: Visualizar Tarefas

1. O usuário solicita acessar a lista de tarefas.
2. O sistema verifica no banco de dados se existe alguma tarefa salva.

3. Se a lista estiver vazia, o sistema exibe: "Você não tem nenhuma tarefa pendente hoje!".
4. Se houver tarefas salvas, o sistema pega a lista e exibe cada tarefa na tela, mostrando título e prioridade.

4. Mapeamento Arquitetural (Preparação para Fase 2)

Estrutura do Repositório:

```
tasktracker-projeto/
├── README.md                # Documentação inicial e apresentação
do projeto
├── .gitignore               # Configuração para ignorar arquivos
desnecessários no versionamento
├── docs/
|   └── planejamento.pdf     # Este documento de planejamento (Fase
1)
└── src/
    └── main.py              # Código fonte principal em Python
(Fase 2)
```

Plano de Aprendizado para Git/GitHub

Na próxima fase do Bootcamp, minha expectativa é entender na prática o conceito de versionamento de código. Quero aprender o que é e como realizar um "commit" para salvar minhas alterações de forma segura, como organizar meu histórico de trabalho e, principalmente, como conectar meu ambiente de desenvolvimento local ao repositório remoto no GitHub, garantindo que o projeto TaskTracker possa ser acessado e executado de qualquer lugar.

---

5. Link da Apresentação em Vídeo

Link do YouTube: [Cole o link do seu vídeo aqui após a gravação]

TaskTracker - Planejamento Lógico (Fase 1)

Entrega Inicial — Bootcamp II

Aluno: Matheus Andrade Naves
Curso: Ciência de Dados e Machine Learning - Asa Norte

Problema e Solução Proposta

Entrega Inicial — Bootcamp II

Aluno: Matheus Andrade Naves
Curso: Ciência de Dados e Machine Learning - Asa Norte

Contexto e Propósito do Projeto

O Problema:

Desorganização no gerenciamento de tarefas diárias em meios informais (papel/mensagens).

Esquecimento de prazos, falta de priorização e sensação de baixa produtividade.

A Solução (TaskTracker):

Sistema digital simples para centralização e acompanhamento de pendências.

Organização por níveis de urgência e marcação de tarefas concluídas.

Estruturação de Dados e Regras do Sistema

Mapeamento de Dados:
Entradas: Título (Obrigatório), Descrição (Opcional), Prioridade e Data de Vencimento.
Saídas: Lista de Tarefas, Mensagens de Sucesso/Erro e Resumo de Concluídas.

Regras de Negócio Implementadas:
Obrigatoriedade: Impede cadastros com campo "Título" em branco.
Restrição de Valores: Campo "Prioridade" aceita apenas Alta, Média ou Baixa.
Validação Temporal: Bloqueia cadastro de datas de vencimento no passado.

Lógica de Funcionamento da Aplicação

Fluxo de Cadastro:
Solicita Título ->Valida preenchimento (Retorna erro se vazio).
Solicita Prioridade -> Armazena registro no sistema.
Exibe mensagem: "Tarefa cadastrada com sucesso!".

Fluxo de Visualização:
Consulta a base de dados do sistema.
Se vazia: Exibe "Você não tem nenhuma tarefa pendente hoje!".
Se preenchida: Exibe a lista detalhada com Título e Prioridade.

Estrutura do Repositório (Preparação para Fase 2)

Organização de Pastas e Arquivos:

```
tasktracker-projeto/
├── README.md              # Documentação e apresentação do projeto
├── .gitignore             # Configuração de arquivos ignorados
├── docs/
│   └── planejamento.pdf   # Planejamento lógico (Fase 1)
└── src/
    └── main.py            # Código fonte principal em Python (Fase 2)
```

Fluxo de Visualização:
Consulta a base de dados do sistema.
Se vazia: Exibe "Você não tem nenhuma tarefa pendente hoje!".
Se preenchida: Exibe a lista detalhada com Título e Prioridade.

Fechamento da Fase 1 e Próximas Etapas

Entregáveis da Fase 1:
Documento de Planejamento Lógico concluído.
Apresentação em vídeo (Pitch explicativo).

Próxima Etapa (Fase 2):
Desenvolvimento da aplicação em Python (src/main.py).
Publicação final no repositório GitHub.